Orchestrator Epistemic Capture

Problem, Definition, and Architectural Solution

Clarence "Professor Bone" Downs
Professor Bone Lab | Johns Hopkins University, Agentic AI Certificate Program
Version 1.2 | Revised February 2026

Abstract

This paper identifies and addresses a governance failure mode in production multi-agent systems that operates at the control plane rather than the data plane: orchestrator epistemic capture.

In a well-designed multi-agent system, every agent is evaluated, every boundary surface is gated, and every memory write is authorized. The data plane is governed. But one component governs all of it and is itself ungoverned: the orchestrator. When that policy engine develops systematic biases through continuous exposure to the agents it governs, it does not produce bad answers. It produces a distorted understanding of what a bad answer is. Every downstream mechanism that depends on the orchestrator's judgment, including the Reflexion reinforcement learning loop, degrades in alignment with that distortion.

This paper develops the formal definition of orchestrator epistemic capture, explains why it is structurally harder to detect than model drift, identifies the three decision types through which it most rapidly accelerates, and proposes a five-mechanism solution anchored by a Meta-evaluation checkpoint positioned outside the orchestrator's epistemic influence.

A post-audit addendum incorporates three verified architectural enhancements and two corrected audit recommendations, including the identification of federated capture as a failure mode introduced by naive distributed orchestration.

Keywords: multi-agent systems, orchestrator governance, epistemic capture, Reflexion, meta-evaluation, shadow policy, audit architecture, KL-divergence, federated capture

---

Part 1: The Problem Surface

1.1 The System

Consider a production multi-agent system consisting of five specialized agents operating in a logistics environment. Each agent executes a defined role: inventory analysis, demand forecasting, compliance validation, reorder recommendation, and execution. Together they form a coordinated pipeline where the output of one agent becomes the input of the next.

Governing this system is an orchestrator, a policy engine that sits above all five agents and manages the control plane. The orchestrator owns every cross-cutting decision:

- Which agent receives which input (ROUTE_DECISION)
- Whether a failed output gets retried or rejected (RETRY_DECISION)
- Whether a failure gets escalated to human review (ESCALATION_DECISION)
- What gets summarized and surfaced to agents for reflection (SUMMARY_EMISSION)
- What gets written to long-term episodic memory (MEMORY_COMMIT_AUTH)
- What gets quarantined pending investigation (QUARANTINE_DECISION)

The agents operate in the data plane. The orchestrator operates in the control plane. This separation is correct by design. It prevents agents from governing themselves and eliminates evaluator capture at the agent level.

1.2 The Reflexion RL Loop

The system implements Reflexion, a verbal reinforcement learning architecture in which agents improve over time not by updating model weights, but by generating natural-language self-reflections after failure and storing them as episodic memory (Shinn et al., 2023).

The loop operates as follows:

```
Agent executes task
→ Evaluator scores the output
→ If failed: Self-Reflector generates verbal diagnosis
→ Reflection committed to episodic memory
→ Next execution retrieves memory as context
→ Agent attempts task again, informed by prior reflection
```

This loop is powerful because it enables continuous self-improvement without retraining. It is consequential for the same reason: the loop depends on the integrity of the evaluation signal. The evaluator's judgment becomes the agent's definition of error. Whatever is committed to episodic memory becomes its model of correctness

The Reflexion loop does not verify the quality of its own inputs. It amplifies them.

1.3 The Evaluation Architecture

To govern this system correctly, three evaluation layers are in place.

Layer 1, Contract Gate (Deterministic). Schema compliance, required fields, type integrity, invariant enforcement. Zero model dependency. Fast, binary, with specific failure reasons.

Layer 2, Behavioral Semantic Scorer. Intent alignment, coherence, risk classification, tool usage validation, and drift scoring. LLM-assisted. Synchronous at mission-critical boundaries.

Layer 3, System Coherence Arbiter. Cross-agent consistency, state divergence detection, contradiction resolution, and temporal stability scoring. Operates at the system level, not the individual agent level.

Each agent is surrounded by deterministic input and output checkpoints it does not own or control. Boundary surfaces between agents have independent validators on both sides of every handoff. Transient commits pass through lighter gates. Persistent commits, those that write to long-term episodic memory, pass through stricter gates. Nothing enters long-term memory without explicit evaluation authorization.

The data plane is well-governed.

1.4 The Gap

In this architecture, every agent is evaluated. Every boundary surface has a checkpoint. Every memory write is gated. The evaluation layers are externalized from the agents. The system looks robust.

But one component governs all of this and is itself ungoverned.

The orchestrator.

Part 2: Orchestrator Epistemic Capture

2.1 Definition

Epistemic, as used here, refers to knowledge: what a system believes to be true and how it forms those beliefs.

Capture describes the condition where a governing body has been compromised by the thing it is supposed to govern, not through malicious intent, but through accumulated structural influence over time.

Orchestrator epistemic capture is the condition where the policy engine governing all agents develops systematic biases in how it interprets, scores, and routes information, and because no layer exists above it to audit those biases, the entire system's understanding of correctness, failure, and ground truth gradually conforms to the orchestrator's distorted model rather than reality.

It is not a single wrong decision. It is the slow installation of a flawed worldview into the architecture itself.

2.2 How It Develops

The orchestrator develops epistemic capture through continuous exposure to the outputs, telemetry, and behavior of the five agents it governs. Over thousands of execution cycles, its policy decisions begin conforming to the patterns the agents produce. It is governing what it has been shaped by.

This is a probabilistic drift, not a discrete failure event. No single decision is obviously wrong. The orchestrator does not malfunction. It gradually normalizes the behavior it observes most frequently, including degraded behavior, and begins treating that normalized pattern as the baseline against which correctness is measured.

Three orchestrator decisions are most likely to accelerate this drift.

SUMMARY_EMISSION. The orchestrator controls what agents reflect on. Biased summarization shapes the entire Reflexion loop. What gets learned, what gets committed to memory, and what gets treated as ground truth are all downstream of what the orchestrator chose to surface. This is the highest-leverage epistemic control point in the system.

ESCALATION_DECISION. When the orchestrator suppresses escalation, humans stop seeing edge cases. The human oversight layer, the only component that operates entirely outside the system's epistemic frame, gets starved of signal. The system appears stable while drifting. Threshold creep here is invisible by design because the mechanism that would surface it has been quieted.

MEMORY_COMMIT_AUTH. The orchestrator decides what persists. What persists becomes future context. What becomes future context shapes agent behavior on every subsequent run. Biased commit authorization is the mechanism by which distorted governance gets permanently installed into the system's long-term memory.

2.3 The Connection to Reflexion

This is where the danger becomes structural.

Reflexion works by agents internalizing failure signals as learned behavior. The agent improves by reflecting on what the evaluator identified as wrong and committing that reflection to episodic memory. On the next run, that memory is retrieved as trusted context. The agent acts on it.

Under orchestrator epistemic capture, the failure signals the orchestrator produces are distorted. Those distorted signals become the input to the Reflexion loop. The agent reflects on them. The agent commits those reflections to memory. The agent retrieves them as ground truth.

The agent is not malfunctioning. It is doing exactly what Reflexion designed it to do. The feedback it is learning from has been corrupted at the source.

The self-improvement loop continues running. It is now systematically improving agents at being wrong in the specific way the orchestrator defined. Reflexion is only as trustworthy as the evaluation signal feeding it. Orchestrator epistemic capture corrupts that signal at the source, turning the self-improvement mechanism into a self-corruption mechanism.

2.4 Why This Is Harder to Detect Than Model Drift

Model drift is visible at the output level. An agent starts producing bad answers. You can measure it, score it, and catch it through the evaluation layer. It lives in the data plane.

Orchestrator epistemic capture operates one level above that. The orchestrator is not producing answers. It is producing definitions of correctness. When the orchestrator drifts, the system does not produce bad answers. It produces a distorted understanding of what a bad answer is.

Every agent in the system learns what good behavior looks like through the feedback it receives from the orchestrator: what passed, what failed, what got committed to memory, what got flagged. That feedback is the system's reality. If the orchestrator's governance decisions are systematically biased, the agents are optimizing against a distorted reality. They are functioning correctly inside a corrupted frame.

Orchestrator epistemic capture is the control-plane version of model drift: not bad answers, but bad governance shaping the definition of correctness.

2.5 The Failure Modes It Produces

Self-confirming governance. The orchestrator routes preferentially to agents whose behavior confirms its existing priors, reinforcing those priors further with each cycle.

Summary bias amplification. Agents only reflect on what the orchestrator chooses to surface. Disconfirming evidence gets filtered. The Reflexion loop becomes an echo chamber.

Escalation suppression. Edge cases stop reaching human reviewers. The human oversight layer loses calibration. The system looks stable from the outside while degrading from within.

Threshold creep. Retry and acceptance thresholds drift incrementally without explicit intent. Each individual adjustment seems reasonable. The cumulative drift is significant and invisible without longitudinal audit.

Part 3: The Architectural Solution

3.1 The Core Principle

The orchestrator cannot evaluate itself. Placing a self-evaluation mechanism inside the orchestrator recreates the capture problem inside the evaluator. The

solution must sit outside the orchestrator's epistemic influence: upstream of the feedback loop, downstream of the orchestrator's decisions, and structurally isolated from the agents the orchestrator governs.

This is a governance infrastructure problem, not a behavioral one. The fix is architectural.

3.2 The Meta-Evaluation Layer

A dedicated checkpoint is positioned between the orchestrator and the feedback loop it governs. Its sole responsibility is to evaluate the orchestrator's governance decisions before they execute and propagate.

```
MAS (5 agents)
→ Orchestrator produces governance decision
→ [Meta-Evaluation Checkpoint]
    → Contract validation of decision structure
    → Policy conformance scoring
    → Shadow policy comparison
→ [PASS] → Decision executes, enters feedback loop
→ [FAIL] → Decision held, audit triggered
```

This checkpoint carries substantially lower epistemic capture risk than the orchestrator for three reasons.

First, its exposure surface is narrow. It observes orchestrator decisions only, not the full complexity of five agents across three evaluation layers. Less exposure means less probabilistic drift.

Second, its contract is tightly bounded. It asks one question: does this orchestrator decision conform to defined governance policy? That is a far smaller evaluation surface than what the orchestrator manages.

Third, the feedback loop cannot influence it. The checkpoint sits upstream of the loop. Agent behavior never reaches it directly. The contamination vector that corrupts the orchestrator does not exist at the checkpoint level.

3.3 The Five Control Mechanisms

Mechanism 1: Policy Trace Contract

Every orchestrator decision emits a structured, immutable record before it executes:

```
decision_id
decision_type
timestamp
input_hash        (pointer to trace, not raw data)
thresholds_applied
chosen_action
alternative_actions_available
confidence_score
policy_version
outcome_tag       (populated at reconciliation time)
```

Raw agent telemetry is never included. Input hashes preserve auditability without exposing sensitive trace data. The outcome tag is populated later during ground truth reconciliation, connecting each governance decision to what actually happened as a result.

Mechanism 2: Shadow Policy Engine

A second policy engine runs in parallel to the orchestrator in shadow mode. It receives the same inputs. It produces an independent governance decision. It never executes; its decisions are for comparison only.

Disagreement between the live orchestrator and the shadow policy triggers a distribution-based audit, not a threshold-based one. A fixed threshold trigger is gameable and becomes stale as the system evolves. Distribution-based triggering watches the shape of disagreement over time. When that distribution shifts, when more disagreements cluster in a specific decision type or disagreement variance narrows suspiciously, that shift triggers audit regardless of whether any single disagreement crossed a fixed line.

Threshold-based detection catches individual anomalies. Distribution-based detection catches systematic drift. Orchestrator epistemic capture is a drift

problem. The detection mechanism must match the failure mode.

Mechanism 3: Human Review of Governance Decisions

A statistically sampled set of orchestrator decisions is periodically surfaced to human reviewers. Not agent outputs. Not evaluation scores. The governance decisions themselves.

Did the retry thresholds make sense given the inputs? Did the summarization choices preserve key failure signal? Did the escalation decisions reflect actual risk, or suppress it?

This is the only mechanism that breaks the closed loop entirely. The human reviewer operates outside the system's epistemic frame. They are the external ground truth anchor that no internal component can provide.

Mechanism 4: Ground Truth Reconciliation

At defined intervals, the orchestrator's committed decisions are reconciled against real-world outcomes: results that the orchestrator never touched, verified externally or by humans.

Did the corrections the orchestrator pushed actually produce better agent behavior? Did the memory it authorized actually improve downstream performance? Divergence between what the orchestrator governed toward and what reality produced is the signal that calibration has drifted.

Mechanism 5: Summarization Audit

Summaries emitted by the orchestrator for agent reflection are evaluated as first-class artifacts before they enter the Reflexion loop.

A coverage test confirms the summary preserved key failure facts. A distortion test confirms it did not omit disconfirming evidence. A leakage test confirms it did not include prohibited telemetry from other agents.

A summary that fails any of these tests does not enter the Reflexion loop. It is held, flagged, and routed to human review.

3.4 Audit Log Architecture

Orchestrator audit logs are stored in an isolated, append-only store with three enforced properties.

Agent-inaccessible. The audit store sits outside all agent memory partitions and outside the orchestrator's own read scope. No agent retrieval path reaches it.

Orchestrator-write-only. The orchestrator emits structured decision records to the audit store at decision time, before the decision executes. It cannot modify or delete prior records.

Human and meta-eval readable. The shadow policy engine and human reviewers have read access through a separate, access-controlled interface that bypasses the orchestrator entirely. The investigation path never routes through the component being investigated.

The audit store is the one component in this architecture with no evaluation gate above it. Its integrity depends entirely on immutability and access control. Governance built on a mutable audit log is governance that can be silently undone.

3.5 Keeping the Checkpoint Aligned

The meta-evaluation checkpoint itself requires governance to prevent its own drift. This is where the solution for the broader system applies recursively.

Human review periodically samples the checkpoint's evaluations of orchestrator decisions, not the agents' outputs, not the orchestrator's outputs, but the checkpoint's own scoring. This keeps the checkpoint calibrated against human-defined policy intent.

Ground truth reconciliation connects the checkpoint's pass/fail decisions on orchestrator governance to real-world outcomes. If the checkpoint is clearing bad orchestrator decisions, that divergence surfaces in reconciliation.

These are not solutions unique to this problem. They are reusable governance primitives that apply at every layer where capture risk exists. The governance

architecture is self-similar: each layer is governed by the same mechanisms used to govern the layers below it.

Architectural Summary

```
External Ground Truth
        |
Human Review + Reconciliation
        |
Meta-Evaluation Checkpoint  <--  Shadow Policy Engine
        |
   Orchestrator
  (Control Plane)
        |
+---------------------------------+
|  5-Agent MAS (Data Plane)       |
|  + 3-Layer Eval Mesh            |
|  + Reflexion RL Loop            |
|  + Episodic Memory              |
+---------------------------------+
        |
Append-Only Audit Log
(Agent-inaccessible, orchestrator-write-only,
 human and meta-eval readable)
```

Every layer has a governance mechanism above it. No component is self-referentially evaluating its own correctness. The closed loop that enables epistemic capture at every level has been broken by positioning independent, deterministic checkpoints upstream of each feedback path, and anchoring the entire stack to human judgment and external ground truth at the top.

The agents cannot govern themselves. The orchestrator cannot govern itself. Governance is infrastructure, not behavior. And infrastructure must be audited from outside the system it governs.

Addendum: Architecture Refinements

Post-Audit Review and Corrections

Addendum Date: February 2026
Basis: External technical audit review, pushback analysis, and citation verification
Status: Verified. All research references confirmed prior to inclusion.

Why This Addendum Exists

Following the initial document, an external technical audit was conducted against the architecture. The audit identified several enhancement recommendations. Those recommendations were then subjected to second-order architectural review: asking not just whether the recommendations were useful, but whether they introduced new failure modes of their own.

Three recommendations survived that review as genuine improvements. Two audit recommendations were corrected before adoption. All citations in the original audit were independently verified and found to be hallucinated, fabricated by the AI system that produced the audit. This addendum incorporates only what passed verification.

This process illustrates a governance principle that applies beyond the specific problem addressed here. Audits of epistemic capture architectures are themselves subject to auditor-level epistemic capture. Even correction mechanisms must be validated from outside the system producing them.

Correction 1: Distributed Orchestration Requires Epistemic Diversity

What the audit recommended. Replace the single orchestrator with distributed micro-orchestrators using consensus-based decision validation to eliminate the single point of governance drift.

Why it was accepted prematurely. Distribution appears to solve centralized capture. It does not. It redistributes it.

The corrected position.

If each micro-orchestrator instance shares the same model lineage, reward heuristics, or reflection bias, their consensus will converge on the same distorted worldview. This produces federated capture: the same epistemic failure distributed across multiple nodes, with the additional problem that it now appears to have been validated by consensus.

Distributed orchestration is only safe if each controller exhibits genuinely independent epistemic grounding. This requires three properties: model heterogeneity, meaning each micro-orchestrator is built from distinct policy priors, fine-tuning data, or evaluator seeds; orthogonal error profiles, meaning independent reflection and correction mechanisms so that bias directions differ structurally rather than superficially; and diversity-weighted aggregation, meaning the meta-evaluation layer does not naively aggregate results but applies diversity weighting analogous to ensemble bagging, discounting correlated outputs.

Without these three properties enforced, distributing the orchestrator does not reduce capture risk. It hides it behind the appearance of consensus.

Revised guidance. Distributed orchestration is conditionally safe. The condition is demonstrated epistemic diversity across all controller instances. If that condition cannot be verified, a single well-governed orchestrator with a strong meta-evaluation checkpoint is safer than distributed orchestrators sharing a common epistemic frame.

> **PACS Implementation Note (2026-03-05):** The federated capture failure mode described in this correction governed the decision to specify the PACS Meta-Evaluation Checkpoint as one unified component rather than three separate checkpoint components, one per protected injection point. Distributing validation across three separate components without demonstrated epistemic diversity between them would not reduce capture risk. It would hide it behind the appearance of independent validation — precisely the failure this correction identifies. One Checkpoint with three internal pathways keeps the epistemic frame unified, the audit regime indivisible, and the governance identity singular. That decision is locked in PACS architecture as of this date.

Correction 2: Formal Policy Verification Is a Partial Guardrail, Not a Solution

What the audit recommended. Add a formal policy verification layer using constraint solvers such as Z3 or Alloy to guarantee conformance of the policy trace contract.

Why it was accepted prematurely. Formal verification sounds comprehensive. Its scope is structurally limited.

The corrected position.

Constraint solvers work on structural properties. They can verify that no decision bypasses the meta-evaluation checkpoint, that policy thresholds are always non-negative, and that no unreachable state exists in the decision graph. These are valuable guarantees.

They cannot verify behavioral alignment. They cannot confirm that an escalation decision reflects actual risk. They cannot ensure that a summarization choice preserved semantic fidelity. Intent, context, and meaning are outside the scope of formal verification.

Revised guidance. Integrate formal verification for structural policy constraints only. Treat it as a contract layer enforcement tool, not a behavioral governance solution. Behavioral conformance remains the responsibility of the shadow policy comparison and human review mechanisms specified in the original architecture.

The addition to the policy trace contract is a formal structural validator that runs deterministically before any decision executes, confirming schema, field presence, threshold bounds, and decision graph reachability. It does not replace semantic evaluation. It runs before it.

Enhancement 1: KL-Divergence Drift Metric for Shadow Policy

What this adds. A quantitative mathematical implementation of the distribution-based audit trigger described in the original document.

The original document stated. Distribution-based triggering watches the shape of disagreement over time. When that distribution shifts, audit triggers.

The operationalized version.

KL-divergence, formally the Kullback-Leibler divergence, measures the difference between two probability distributions. Applied here, it continuously

measures the divergence between the live orchestrator's decision distribution and the shadow policy engine's decision distribution over a rolling window of executions.

```
KL(P || Q) = sum over x of P(x) log( P(x) / Q(x) )

Where:
P = live orchestrator decision distribution
Q = shadow policy decision distribution
Window = rolling N executions (N calibrated per deployment)
```

When KL-divergence remains low, the two policy engines are making similar decisions. The orchestrator is behaving consistently with its defined policy baseline. When KL-divergence rises steadily over time, it signals systematic drift: the orchestrator's decisions are diverging from the shadow in a directional, non-random way that indicates developing bias rather than normal variance.

This is superior to a fixed threshold trigger for three reasons. It detects drift direction, not just magnitude. It is resistant to gaming, because an orchestrator cannot pass a KL-divergence test by occasionally agreeing with the shadow while systematically disagreeing on specific decision types. And it surfaces threshold creep as a distribution shift before any individual decision appears anomalous.

Integration point. The KL-divergence metric feeds directly into the meta-evaluation checkpoint as a continuous signal. A rising KL trend below the audit trigger threshold generates a warning. A KL value crossing the calibrated threshold triggers a full shadow policy disagreement audit automatically.

Enhancement 2: Tamper-Evident Audit Log Backend

What this adds. Cryptographic immutability to the append-only audit store, converting it from operationally immutable to verifiably immutable.

The original document stated. The audit store is append-only and immutable. Its integrity depends entirely on immutability and access control.

The gap. Operational immutability, enforced by access controls and write policies, can be circumvented by sufficiently privileged actors or infrastructure failures. It provides no forensic proof that a log has not been altered.

The enhancement.

A tamper-evident ledger backend applies cryptographic hashing to each audit record at write time. Each record includes the hash of the previous record, forming a hash chain. Any alteration to any prior record invalidates all subsequent hashes, making tampering detectable without requiring access to the original data.

```
Record N:
  content: { decision_id, decision_type, timestamp, input_hash,
... }
  record_hash: SHA256(content + previous_record_hash)

Record N+1:
  content: { ... }
  record_hash: SHA256(content + record_N_hash)
```

This means deletion is detectable, because removing a record breaks the hash chain. Modification is detectable, because altering any field invalidates the record hash and all subsequent hashes. Insertion is detectable, because inserting a record between existing records breaks chain continuity.

Implementation options include AWS QLDB, which provides a managed immutable ledger with cryptographic verification, or a self-managed Merkle tree structure for environments requiring full infrastructure control.

Why this matters for governance. The audit store is the forensic foundation of the entire meta-evaluation architecture. If it can be silently altered, every

governance guarantee built on top of it is undermined. Cryptographic immutability converts the audit log from a record that should not have been altered to a record that can be proven not to have been altered. That distinction is the difference between policy and evidence.

Enhancement 3: External World Feedback Loop for Ground Truth Reconciliation

What this adds. A concrete operational mechanism for anchoring ground truth reconciliation to real-world outcomes rather than internal system metrics.

The original document stated. At defined intervals, the orchestrator's committed decisions are reconciled against real-world outcomes, results that the orchestrator never touched, verified externally or by humans.

The gap. The original document established the principle but did not specify the data source or reconciliation mechanism.

The enhancement.

In a logistics MAS, real-world outcomes are directly measurable and exist independently of the system's internal state. Inventory accuracy, order fulfillment rates, compliance audit results, and delivery performance are ground truth signals the orchestrator cannot manipulate because they originate outside the system.

The external feedback loop operates as follows:

```
Orchestrator commits governance decision
→ Decision tagged with outcome_id at write time
→ System executes based on decision
→ Real-world outcome recorded independently
   (inventory audit, fulfillment confirmation, compliance check)
```

```
→ Reconciliation process matches outcome_id to real-world result
→ Outcome tag populated in audit record
→ Divergence between governed behavior and real outcome scored
→ Divergence score feeds meta-evaluation checkpoint calibration
```

The reconciliation process does not route through the orchestrator. It runs as an independent service that reads from the audit store and from the external outcome data source directly. The orchestrator cannot influence what the reconciliation process sees or how it scores divergence.

Over time, systematic divergence between orchestrator governance decisions and real-world outcomes is the most reliable signal that epistemic capture has occurred. Real-world outcomes are the one ground truth the orchestrator's internal biases cannot redefine.

Calibration use. Divergence scores from reconciliation feed back into the meta-evaluation checkpoint's policy conformance thresholds, tightening or relaxing them based on demonstrated real-world performance rather than internal system agreement. This is the mechanism by which the entire governance stack stays anchored to reality rather than to an increasingly self-referential model of correctness.

Citation Protocol

All research references in this document follow this classification system.

[VERIFIED]: Confirmed via DOI, arXiv identifier, or official conference proceedings. Safe to cite in the Build Guide.

[RESEARCH LEAD]: Conceptually supported by active research area but specific citation not independently confirmed. Use as a search direction only. Do not cite as a source.

[UNVERIFIED, DO NOT CITE]: Appeared in external audit or secondary source but could not be confirmed. Treat as hallucinated until proven otherwise.

References used in this document:

- Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS 2023, arXiv:2303.11366. **[VERIFIED]**
- KL-divergence as drift detection metric. **[RESEARCH LEAD]** Applied in ML monitoring literature broadly; specific MAS governance application is original to this architecture.
- Tamper-evident ledger backends (AWS QLDB, Merkle chain structures). **[VERIFIED]** Production infrastructure, independently confirmable.
- "Reflexion++ ICLR 2024," "Yin et al. AAAI 2025," "ArXiv:2501.1179." **[UNVERIFIED, DO NOT CITE]** All three confirmed hallucinated via independent search.

Updated Architectural Summary (v1.2)

```
External Ground Truth (Real-World Outcomes)
        |
Reconciliation Service (Independent, Orchestrator-blind)
        |
Human Review <---> KL-Divergence Monitor
        |
Meta-Evaluation Checkpoint
  + Formal Structural Validator (Contract Layer)
  + Shadow Policy Engine (Epistemically Diverse)
        |
   Orchestrator (Control Plane)
  [Distributed only if epistemic diversity enforced]
        |
+-------------------------------------+
```

```
|  5-Agent MAS (Data Plane)           |
|  + 3-Layer Eval Mesh                |
|  + Reflexion RL Loop                |
|  + Episodic Memory (Gated)          |
+-------------------------------------+
        |
Tamper-Evident Audit Log
(Append-only, cryptographically immutable,
 agent-inaccessible, orchestrator-write-only,
 human and meta-eval readable)
```

The three enhancements strengthen the original architecture at its three weakest points. The audit trigger lacked mathematical precision; it is now operationalized via KL-divergence. The audit log lacked forensic proof of integrity; it is now cryptographically immutable. Ground truth reconciliation lacked an operational data source; it is now anchored to real-world logistics outcomes independent of the orchestrator's epistemic frame.

The two corrections prevent the architecture from introducing new failure modes in the process of mitigating existing ones. Distributed orchestration is conditionally safe, not universally safe. Formal verification is a structural guardrail, not a behavioral governance solution.

Professor Bone Lab
Version 1.2, Revised February 2026
Classification: Architect-grade reference. Build Guide integration pending.
All citations verified independently before inclusion.